

Data-oriented Programming

Übungen

Authentication: Teil 1

Implementiere die Methode `validate` mit Guards inklusive den folgenden Bedingungen für die folgende Varianten für `AuthToken`:

- Wenn das `password` in `PasswordToken` kürzer als 8 Zeichen ist, soll `ValidationResult.WEAK` zurückgegeben werden, andernfalls `ValidationResult.OK`.
- Wenn das `OAuthToken` abgelaufen ist, d. h., `expiresAt` ist vor `Instant.now()`, soll `ValidationResult.EXPIRED` zurückgegeben werden, andernfalls `ValidationResult.OK`.
- In allen anderen Fällen soll `ValidationResult.INVALID` zurückgegeben werden.

```
class Authentication {  
  
    interface AuthToken {}  
  
    record PasswordToken(String username, char[] password) implements AuthToken {}  
    record OAuthToken(String provider, String accessToken, Instant expiresAt) implements AuthToken {}  
  
    enum ValidationResult { OK, EXPIRED, WEAK, MALFORMED, INVALID }  
  
    ValidationResult validate(AuthToken authToken) {  
        return null; // TODO  
    }  
}
```

Authentication: Teil 1 Lösung

```
interface AuthToken {}
```

```
record PasswordToken(String username, char[] password) implements AuthToken {}
```

```
record OAuthToken(String provider, String accessToken, Instant expiresAt) implements AuthToken {}
```

```
enum ValidationResult { OK, EXPIRED, WEAK, MALFORMED, INVALID }
```

```
class Authentication {
```

```
    ValidationResult validate(AuthToken authToken) {
```

```
        return switch (authToken) {
```

```
            case PasswordToken pt when pt.password().length < 8 -> ValidationResult.WEAK;
```

```
            case PasswordToken _ -> ValidationResult.OK;
```

```
            case OAuthToken ot when ot.expiresAt().isBefore(Instant.now()) -> ValidationResult.EXPIRED;
```

```
            case OAuthToken _ -> ValidationResult.OK;
```

```
            default -> ValidationResult.INVALID;
```

```
        };
```

```
    }
```

```
}
```

Authentication: Teil 2

Falls du deine Lösung von “Authentication Teil 1” mit default implementiert hast, ändere sie jetzt so ab, dass sie kein default verwendet und trotzdem Exhaustiveness sicherstellt.

Authentication: Teil 2 Lösung

```
sealed interface AuthToken permits PasswordToken, OAuthToken {}
```

```
record PasswordToken(String username, char[] password) implements AuthToken {}
```

```
record OAuthToken(String provider, String accessToken, Instant expiresAt) implements AuthToken {}
```

```
enum ValidationResult { OK, EXPIRED, WEAK, MALFORMED, INVALID }
```

```
class Authentication {
```

```
    ValidationResult validate(AuthToken authToken) {
```

```
        return switch (authToken) {
```

```
            case PasswordToken pt when pt.password().length < 8 -> ValidationResult.WEAK;
```

```
            case PasswordToken _ -> ValidationResult.OK;
```

```
            case OAuthToken ot when ot.expiresAt().isBefore(Instant.now()) -> ValidationResult.EXPIRED;
```

```
            case OAuthToken _ -> ValidationResult.OK;
```

```
            case null -> ValidationResult.INVALID;
```

```
        };
```

```
    }
```

```
}
```

Payment Fee: Teil 1

Implementiere die Methode `fee`, die je nach Zahlungsart folgende Gebühren zurückgibt:

- BankTransfer: fix 25 Cent (unabhängig vom Wert in `amountInEuro`)
- CreditCard: 1.5 % von `amountInEuro`
- Falls `amountInEuro` kleiner 0 ist, soll eine `IllegalArgumentException` geworfen werden (beachte das Fail Fast Principle).

Die Gebühren werden von `fee` als Euro-Betrag zurückgeben.

Stelle Exhaustiveness sicher. Ändere hierzu den vergebenen Code.

```
class PaymentFee {  
  
    interface Payment {}  
  
    record CreditCard(String number) implements Payment {}  
    record BankTransfer(String iban) implements Payment {}  
  
    double fee(Payment payment, double amountInEuro) {  
        return -1; // TODO  
    }  
}
```

Payment Fee: Teil 1 Lösung

```
sealed interface Payment permits CreditCard, BankTransfer {}

record CreditCard(String network, String panMasked) implements Payment {}
record BankTransfer(String iban) implements Payment {}

class PaymentFee {

    double fee(Payment payment, int amountInEuro) {
        if (amountInEuro < 0) {
            throw new IllegalArgumentException("amountInEuro darf nicht negativ sein");
        }

        return switch (payment) {
            case BankTransfer _ -> 0.25;
            case CreditCard _ -> amountInEuro * 0.015;
        };
    }
}
```

Payment Fee: Teil 2

Füge zu deiner Lösung von „Payment Fee: Teil 1“ eine neue Zahlungsart Crypto hinzu:

```
record Crypto(String id) implements Payment {}
```

Die Gebühr für Crypto beträgt 1 % von amountInEuro, wobei **1 Cent Mindestgebühr** anfällt.

Zusatzfrage:

Welche Auswirkung hätte eine default-Klausel auf die Exhaustiveness?

Payment Fee: Teil 2 Lösung

```
sealed interface Payment permits CreditCard, BankTransfer, Crypto {}

record CreditCard(String network, String panMasked) implements Payment {}
record BankTransfer(String iban) implements Payment {}
record Crypto(String id) implements Payment {}

class PaymentFee {

    double fee(Payment payment, int amountInEuro) {
        if (amountInEuro < 0) {
            throw new IllegalArgumentException("amountInEuro darf nicht negativ sein");
        }

        return switch (payment) {
            case BankTransfer _ -> 0.25;
            case CreditCard _ -> amountInEuro * 0.015;
            case Crypto _ -> Math.max(amountInEuro * 0.01, 0.01);
        };
    }
}
```

Zusatzfrage:

Ein switch gilt als exhaustiv, sobald ein default vorhanden ist — auch wenn einzelne konkrete Fälle fehlen. Dadurch verliert man die Kompiler-Garantie, dass alle bekannten Subtypen explizit behandelt werden.

Fügt man z. B. Crypto hinzu, zwingt der Kompiler bei switch ohne default, die fehlenden cases zu implementieren.

Mit default läuft stattdessen alles weiter und neue Typen werden unauffällig in den default-Pfad geschoben → potenziell stille Fehler.

Payment Fee: Teil 3

Schreibe deine Lösung „Payment Fee: Teil 3“ in eine objektorientierte Variante um, sodass Polymorphie verwendet wird.

Zusatzfrage:

Wann wäre eine objektorientierte und wann eine datenorientierte Variante geeigneter in Bezug auf die Erweiterbarkeit des Codes?

Payment Fee: Teil 3 Lösung

```
interface Payment {
    double fee(double amountInEuro);
}

final class PaymentUtil {
    static void assertValidAmount(double amountInEuro) {
        if (amountInEuro < 0.0) throw new IllegalArgumentException("amountInEuro darf nicht negativ sein");
    }
}

record CreditCard(String number) implements Payment {
    @Override
    public double fee(double amountInEuro) {
        PaymentUtil.assertValidAmount(amountInEuro);
        return 0.015 * amountInEuro;
    }
}

record BankTransfer(String iban) implements Payment {
    @Override
    public double fee(double amountInEuro) {
        PaymentUtil.assertValidAmount(amountInEuro);
        return 0.25;
    }
}

record Crypto(String id) implements Payment {
    @Override
    public double fee(double amountInEuro) {
        PaymentUtil.assertValidAmount(amountInEuro);
        return Math.max(0.01, 0.01 * amountInEuro);
    }
}

class PaymentFee {
    double fee(Payment payment, double amountInEuro) {
        return payment.fee(amountInEuro);
    }
}
```

Objektorientiert:

Wenn die Logik für die Gebühren in einzelne Typen gekapselt werden soll (Single Responsibility).
Wenn neue Zahlungsarten häufig hinzukommen.
Neue Zahlungsarten müssen lediglich Payment implementieren und ihre eigene fee-Logik bereitstellen.

Datenorientiert:

Wenn die Logik für die Gebühren zentral in einer Methode (Operation) gekapselt werden soll.
Wenn unterschiedliche Berechnungen für die Gebühren häufig hinzukommen (z. B. pro Country).
Neue Berechnungen für die Gebühren müssen lediglich jede Zahlungsart als case implementieren.